Learning to Get Back Up: Fall Recovery for the MuJoCo Ant

CS 839 Programming Assignment · [TODO: your name] · Fall 2026

1. The behavior (self-righting)

Behavior. After a fall, from a random tipped-over pose (on its side or upside down, sometimes dropped from a height while spinning, on flat, bumpy or sloped ground, and possibly shoved), the Ant must get back onto its feet and **stay** standing still.

Why. Fall recovery is a prerequisite for autonomous legged robots deployed in inspection, mining and search and rescue, where a fall otherwise requires a human to walk into a hazardous area to stand the robot up. Learned recovery is an active topic (e.g. FR-Net, 2025) because scripted get-up motions fail on unusual poses and terrain.

Why it is non-trivial for the Ant. The Ant's hips only swing in the torso plane (±30°) and its knees only fold one way, so when inverted its legs point at the sky and the torso is a sphere resting on the ground: it must use leg inertia and brief ground contacts to roll itself over, which is a very different, impulsive contact pattern from walking, and it must then stop and stabilize. Doing nothing never succeeds (0% success; 8% only when dropped from 1–1.5 m, by landing feet-first by luck), and random torques never succeed either (§3, Table 2).

2. The MDP

All of the MDP is in `robot_recovery/ant_recovery.py` (class `AntRecoveryEnv`, parameters in `RecoveryConfig`); the environment used for all final results is `AntRecoveryLowCost-v0`. It subclasses Gymnasium's `MujocoEnv` and uses the unmodified Ant body and actuators from Gymnasium's `ant.xml`, plus a heightfield terrain strip (`robot_recovery/assets/ant_recovery.xml`). Gymnasium 0.29.1 only provides `Ant-v4`, so v4 assets are used (the fallback permitted by the assignment).

Action space. Unchanged: $a \in [-1, 1]^8$, one torque command per hinge (gear 150), clipped and multiplied by a per-episode motor-strength factor.

State (observation) space. 33 dimensions, restricted to quantities a real robot can measure or estimate: gravity direction in the torso frame (3, as an IMU reports it), torso angular velocity (3), torso linear velocity in the torso frame (3), joint angles (8), joint velocities ×0.1 (8), and the previous action (8). Unlike the default Ant observation, it contains the torso orientation in a frame-independent form, and it omits the absolute height and global position, which a real robot does not know.

Is it Markov? No. The underlying simulator state is the full position and velocity of all bodies **plus** the episode's hidden parameters: terrain shape under the robot, friction, payload mass, motor strength, and whether and when a shove will occur. The observation omits torso height above the ground, the terrain, and these parameters, so two identical observations can have different futures (e.g. the same pose on ice vs. on high-friction ground). The previous action is included to restore some short-term information. The process is therefore a POMDP; the policies are memoryless, which is a deliberate choice that matches what a deployed proprioceptive controller would see.

Initial-state distribution (`reset_model`). Each episode:

- **Terrain:** flat ground (p=0.5), bumpy ground (p=0.25; three 4×4 m tiles with smooth bumps up to 6, 10 or 14 cm), or a slope (p=0.25; three pyramids with 5°, 10°, 15° faces, landing 0.6–1.4 m from the apex, facing any direction).
- **Pose:** random heading, then tilted by U(120°, 180°) about a random horizontal axis; joint angles uniform within their limits; placed 2 cm above the highest ground point within 1.2 m.
- **Drop:** with p=0.3 an extra U(0.3, 1.0) m of height and an angular velocity U(−3, 3) rad/s per axis.
- **Domain randomization:** sliding friction ×U(0.8, 1.2), torso mass ×U(0.9, 1.3), motor strength ×U(0.9, 1.1).
- **Disturbance:** with p=0.5 one shove at a random step in [40, 300): a horizontal velocity change of U(0.3, 1.0) m/s in a random direction plus U(−2, 2) rad/s angular velocity per axis.

Reward (dense shaping; **not** the performance metric). With c = torso up-axis · terrain normal, h = torso height above the terrain, ω = torso angular speed:

$$r = \left(\frac{c+1}{2}\right)^2 + 0.5 \cdot \mathrm{clip}\left(\frac{h-0.25}{0.3}, 0, 1\right)\mathbb{1}[c > 0.7] + 0.5 \cdot \mathbb{1}[c > 0.9]e^{-\omega^2/2} - 0.005\ \|a\|^2 - 0.01\ \|a - a_{\text{prev}}\|^2\ .$$

The terms reward being upright, standing tall (the torso only rises above its 0.25 m radius when the legs carry it), and being still once upright, and penalize effort and jerky actions (smoothness matters on real motors). `AntRecovery-v0` uses twice these two cost weights (0.01, 0.02); halving them was the single most important change for PPO (§5).

Episode length and termination. 400 steps of $\Delta t = 0.05$ s (20 s; physics at 100 Hz, 5 sub-steps per action). The default Ant "healthy height" termination is removed, since it would end every episode immediately for a fallen robot. The only termination is a non-finite simulator state (never observed). Episodes end by time-limit truncation; note that CleanRL's SAC treats truncation correctly (bootstraps), while CleanRL's PPO treats it as a terminal state.

3. True performance metric

Success (binary, per episode): the Ant is standing upright and steady for the **entire final 2.5 s** (last 50 steps) of the episode. "Standing upright and steady" at a step means all of:

1. $c \geq 0.9$: torso up-axis within 25.8° of the local terrain normal (finite differences of the heightfield over ±0.15 m at the spawn point; $(0, 0, 1)$ on flat ground and rubble);
2. $h \geq 0.32$ m: torso centre at least 0.32 m above the terrain directly below it (lying on the torso gives ≈0.25 m; relaxed standing ≈0.40 m);
3. $\omega \leq 1$ rad/s: torso angular speed.

We report the **success rate** (mean over episodes). Secondary: **time to recover**, the first time (s) the condition holds for 1 s continuously (20 s if never). The metric captures exactly the goal (ending on its feet, upright and stable) and is interpretable without the reward: "the Ant recovered in 95% of falls". It cannot be satisfied by partial progress the reward pays for (being half-rolled, standing up briefly and falling again, or rocking while upright), and it requires **staying** up after any shove.

4. Learning curves (PPO vs. SAC)

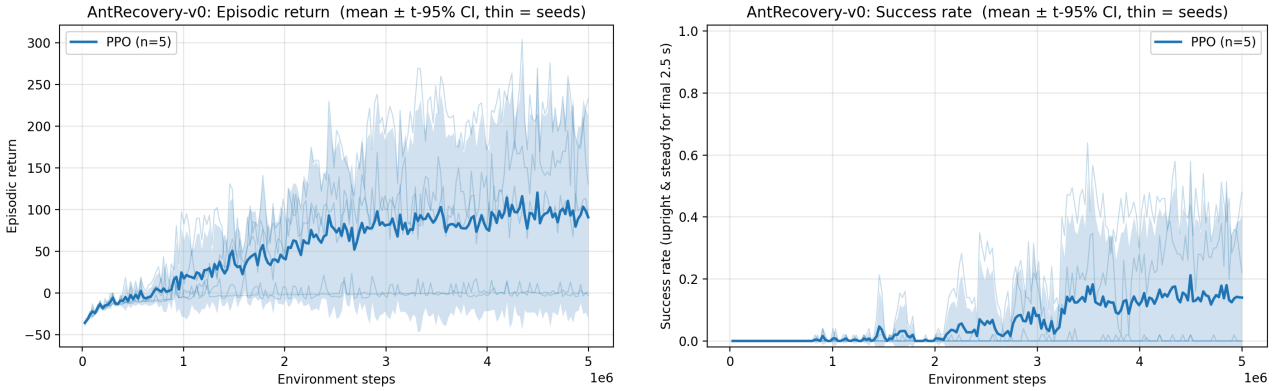


Figure 1: Episodic return (left) and success rate (right) vs. environment steps on `AntRecoveryLowCost-v0`. Bold: mean over 5 paired seeds (1–5 for both algorithms); band: Student-t 95% interval $\bar{x} \pm t_{0.975,4}s/\sqrt{5} = \bar{x} \pm 2.78s/\sqrt{5}$; thin lines: individual seeds. SAC was trained for 1M steps, PPO for 5M.

Aggregation. Each run logs one value per finished training episode (stochastic policy) at irregular steps. For each seed we smooth with a trailing 50-episode moving average, then linearly interpolate onto a common grid of 200

evenly spaced environment steps, and only then average across seeds (smooth-then-aggregate). Grid points before a run's first finished episode are left empty. Non-overlapping intervals are not interpreted as significance tests.

[TODO: final numbers once SAC finishes: SAC success at 1M = ...% (per seed ...); PPO at 1M = 0% (0, 2, 0, 0, 0), at 5M = 14% (46, 23, 0, 0, 0)]

5. Comparison and tuning

[TODO: discussion after final SAC results; draft points: (1) final performance; (2) sample efficiency, success at equal steps; (3) seed variance (PPO bimodal 2/5 learn); explanations: off-policy replay of rare successful flips, 1 gradient update per env step vs. 10 epochs per 2048-step batch then discarded; entropy-regularized exploration vs. PPO collapsing to the zero-effort "lie on the back" optimum; PPO treating time-limit truncation as terminal]

Tuning. All tuning runs used 5M steps for PPO and are in `chtc/batches/ppotune*.txt`. Success at 5M steps (training episodes, mean of last 100), mean (per-seed):

Reward	Envs × steps	Entropy	Width	Success at 5M
`v0` (costs 0.01/0.02)	1×2048	0	64	0% (0, 0)
`v0`	1×2048	0.01	64	4% (8, 0)
`v0`	16×256	0 / 0.01	64	0% (all four seeds)
low-cost	**1×2048**	**0**	**64**	**35% (46, 23)** ← chosen = CleanRL defaults
low-cost	1×2048	0.01	64	0% (0, 0)
low-cost	16×256	0	64	24% (49, 0)
low-cost	16×256	0.01	64	10% (17, 2)
low-cost	1×2048	0	256	17% (0, 23, 27)
low-cost	1×2048	0.01	256	0% (0, 0, 0)
low-cost	16×256	0	256	20% (59, 0, 0)
low-cost	16×256	0.01	256	8% (0, 25, 0)

What worked: halving the control and action-rate costs (with the original costs the cheapest behavior is to lie still on the back; PPO never escaped it, SAC reached 88% vs. 95% with lower costs). What did not: an entropy bonus (0.01), 16 parallel environments with shorter rollouts, and 4× wider networks (256 units, as in SAC). SAC was not tuned beyond the cost change (CleanRL defaults).

6. Robustness beyond the training distribution

[TODO: eval table (100 episodes per condition, deterministic policies, same seeds for all) + stress-ladder figure; rubble field never seen in training]

7. Video

[TODO: `videos/best_sac.mp4`: best SAC seed, several recoveries; highlight reel (supplementary)]

8. Code and reproduction

[TODO: README with exact commands; list of CleanRL changes]

9. Use of AI

[TODO: student to write/approve: Claude Code (Anthropic) was used for ...]